

```

import torch
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

# Convert images to tensors and normalize them
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Load training dataset
train_dataset = torchvision.datasets.FashionMNIST(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

# Load testing dataset
test_dataset = torchvision.datasets.FashionMNIST(
    root='./data',
    train=False,
    download=True,
    transform=transform
)

# Create DataLoaders
train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=64,
    shuffle=False
)

print("Training samples:", len(train_dataset))
print("Testing samples:", len(test_dataset))

```

100%|██████████| 26.4M/26.4M [00:01<00:00, 14.6MB/s]
 100%|██████████| 29.5k/29.5k [00:00<00:00, 232kB/s]
 100%|██████████| 4.42M/4.42M [00:01<00:00, 4.30MB/s]
 100%|██████████| 5.15k/5.15k [00:00<00:00, 23.6MB/s] Training samples: 60000
 Testing samples: 10000

```

import torch.nn as nn
import torch.nn.functional as F

class CNN(nn.Module):

    def __init__(self):
        super(CNN, self).__init__()

        # First convolution layer
        self.conv1 = nn.Conv2d(
            in_channels=1,
            out_channels=32,
            kernel_size=3,
            padding=1
        )

        # Second convolution layer

```

```

        self.conv2 = nn.Conv2d(
            in_channels=32,
            out_channels=64,
            kernel_size=3,
            padding=1
        )

        # Fully connected layers
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):

        # Convolution + ReLU + Max Pooling
        x = F.relu(self.conv1(x))
        x = F.max_pool2d(x, 2)

        # Second convolution + ReLU + Max Pooling
        x = F.relu(self.conv2(x))
        x = F.max_pool2d(x, 2)

        # Flatten
        x = x.view(x.size(0), -1)

        # Fully connected layer
        x = F.relu(self.fc1(x))

        # Output layer
        x = self.fc2(x)

        return x

# Create model
model = CNN()

print(model)

```

```

CNN(
  (conv1): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (fc1): Linear(in_features=3136, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)

```

```

import torch
import torch.optim as optim

# Select device
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

model = CNN().to(device)

# Loss function
criterion = nn.CrossEntropyLoss()

# Optimizer
optimizer = optim.Adam(
    model.parameters(),
    lr=0.001
)

# Number of epochs
epochs = 10

# Store results
train_losses = []

```

```

train_accuracies = []

for epoch in range(epochs):

    model.train()

    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:

        images = images.to(device)
        labels = labels.to(device)

        # Clear previous gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)

        # Calculate loss
        loss = criterion(outputs, labels)

        # Backpropagation
        loss.backward()

        # Update weights
        optimizer.step()

        running_loss += loss.item()

        # Calculate accuracy
        _, predicted = torch.max(outputs.data, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    epoch_loss = running_loss / len(train_loader)
    epoch_accuracy = 100 * correct / total

    train_losses.append(epoch_loss)
    train_accuracies.append(epoch_accuracy)

    print(
        f"Epoch [{epoch+1}/{epochs}] "
        f"Loss: {epoch_loss:.4f} "
        f"Accuracy: {epoch_accuracy:.2f}%"
    )

```

```

Epoch [1/10] Loss: 0.4313 Accuracy: 84.56%
Epoch [2/10] Loss: 0.2765 Accuracy: 89.84%
Epoch [3/10] Loss: 0.2317 Accuracy: 91.38%
Epoch [4/10] Loss: 0.2002 Accuracy: 92.59%
Epoch [5/10] Loss: 0.1724 Accuracy: 93.64%
Epoch [6/10] Loss: 0.1517 Accuracy: 94.42%
Epoch [7/10] Loss: 0.1293 Accuracy: 95.06%
Epoch [8/10] Loss: 0.1111 Accuracy: 95.85%
Epoch [9/10] Loss: 0.0940 Accuracy: 96.47%
Epoch [10/10] Loss: 0.0789 Accuracy: 97.07%

```

```

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)

```

```

model.eval()

all_labels = []
all_predictions = []

with torch.no_grad():

    for images, labels in test_loader:

        images = images.to(device)

        outputs = model(images)

        _, predictions = torch.max(outputs, 1)

        all_labels.extend(labels.numpy())
        all_predictions.extend(
            predictions.cpu().numpy()
        )

# Calculate metrics
accuracy = accuracy_score(
    all_labels,
    all_predictions
)

precision = precision_score(
    all_labels,
    all_predictions,
    average='weighted'
)

recall = recall_score(
    all_labels,
    all_predictions,
    average='weighted'
)

f1 = f1_score(
    all_labels,
    all_predictions,
    average='weighted'
)

print("Test Accuracy :", accuracy * 100, "%")
print("Precision      :", precision)
print("Recall         :", recall)
print("F1 Score       :", f1)

# Confusion matrix
cm = confusion_matrix(
    all_labels,
    all_predictions
)

print("Confusion Matrix:")
print(cm)

```

```

Test Accuracy : 92.07 %
Precision      : 0.9211012286672703
Recall         : 0.9207
F1 Score       : 0.9205713048249791
Confusion Matrix:
[[907  1  21  5  3  1  59  0  3  0]
 [ 1 981  1 12  0  0  4  0  1  0]
 [ 17  1 891  4 46  0 41  0  0  0]
 [ 27  3 16 887 22  0 43  0  2  0]
 [  1  1 39 20 887  0 51  0  1  0]
 [  0  0  0  0  0 983  0 14  0  3]
[128  1 55 12 60  0 740  0  4  0]

```

```
[ 0 0 0 0 0 6 0 978 0 16]
[ 4 1 0 2 0 1 4 2 986 0]
[ 0 0 0 0 0 5 1 27 0 967]]
```

```
import matplotlib.pyplot as plt

classes = [
    'T-shirt/top',
    'Trouser',
    'Pullover',
    'Dress',
    'Coat',
    'Sandal',
    'Shirt',
    'Sneaker',
    'Bag',
    'Ankle boot'
]

model.eval()

images, labels = next(iter(test_loader))

images = images.to(device)

with torch.no_grad():
    outputs = model(images)
    _, predictions = torch.max(outputs, 1)

plt.figure(figsize=(12, 6))

for i in range(10):

    plt.subplot(2, 5, i + 1)

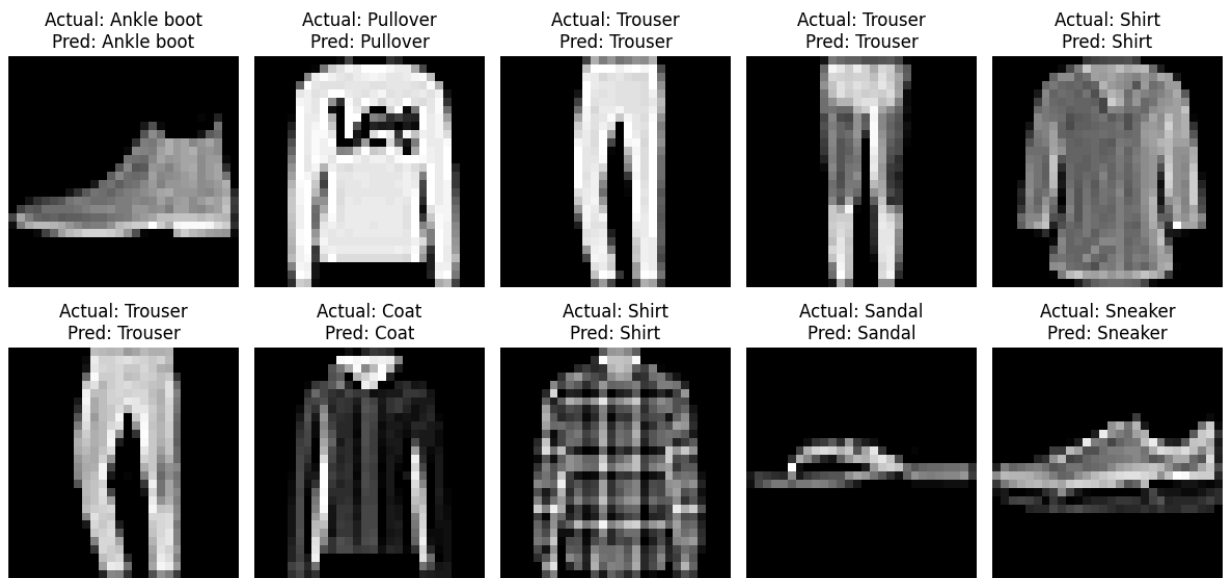
    image = images[i].cpu().squeeze()

    plt.imshow(image, cmap='gray')

    plt.title(
        f"Actual: {classes[labels[i]]}\n"
        f"Pred: {classes[predictions[i]]}"
    )

    plt.axis('off')

plt.tight_layout()
plt.show()
```



```
import seaborn as sns

plt.figure(figsize=(10, 8))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=classes,
    yticklabels=classes
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("FashionMNIST Confusion Matrix")
plt.show()
```

